CS 489/689 Assignment 4: Skin-Lesion Segmentation

Overview

Skin cancer includes melanoma and non-melanoma lesions that appear on the skin. Early detection helps guide clinical assessment and treatment. In this assignment, you will perform **binary skin-lesion segmentation** (*lesion* vs. *background*) using per-pixel masks.

This is a shorter assignment than the previous segmentation project. Instead of many model settings and cross-validation, this assignment focuses on three main model variants:

1. **U-Net trained from scratch**
2. **ResUNet with a pretrained ResNet encoder (encoder frozen)**
3. **ResUNet with a pretrained ResNet encoder (fine-tune all)**

You will use the provided notebook starter code, complete the required `TODO` parts, train the models, and compare their performance. In addition, you will compare two loss functions on the fine-tuned ResUNet model:

- Dice loss
- Dice + Binary Cross-Entropy (Dice+BCE)

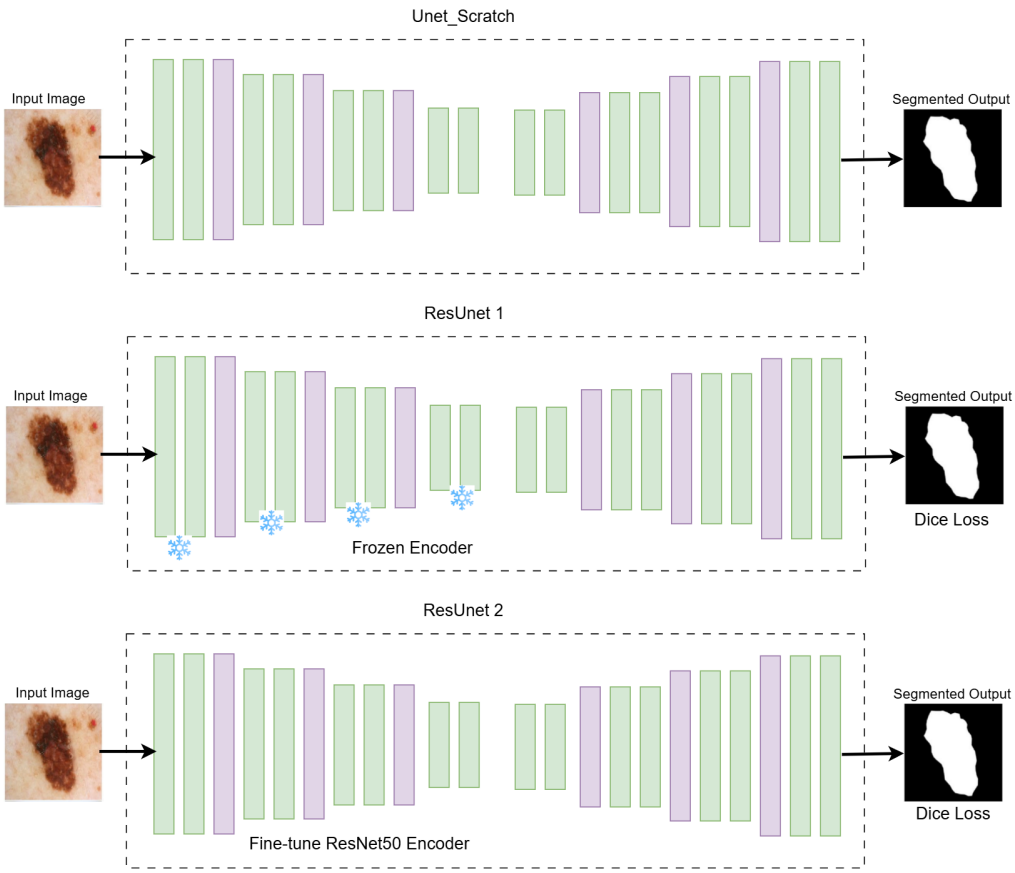

Figure 1: The three required segmentation model variants used in this assignment.

Dataset Access

The dataset is hosted on Google Drive. Download it directly to your account (for example, your RebelX workspace) using:

```
pip install gdown
gdown 1yQhfmxethE6I36Zsqm2jymftVKFk1ias
```

Unzip the archive and place the data in the folder expected by the notebook. Keep the provided folder structure unchanged.

Starter Notebook and What You Need to Implement

The notebook already includes:

- dataset reading,
- preprocessing,
- helper visualization code,
- and some non-essential utility code.

You are responsible for completing the main deep learning parts marked with `TODO`. These may include:

- Dice loss,
- evaluation metrics,
- U-Net forward pass,
- ResUNet forward pass,
- freezing or fine-tuning the encoder,
- training and evaluation code.

Read the notebook comments carefully. They explain what each part should do, what the input is, and what the output should be.

Configuration Settings

The notebook is designed so that you can easily modify training settings from one place. You may change settings such as:

- number of epochs,
- batch size,
- learning rate,
- image size,
- threshold for converting probabilities to binary masks.

Make sure your final reported results are clearly labeled with the settings you used.

Model Variants

Let **ResUNet** denote a U-Net style decoder with a **ResNet-50** encoder.

1. **U-Net (from scratch):** train the entire model from scratch.
2. **ResUNet (encoder frozen):** use a pretrained ResNet encoder and freeze its weights during training.
3. **ResUNet (fine-tune all):** use a pretrained ResNet encoder and train both the encoder and decoder.

Training and Evaluation Requirements

- Use one fixed input size for all experiments within the same comparison.
- Use one normalization strategy for all experiments.
- Use the same train/validation/test split for all models.
- Use the same threshold to convert predicted probabilities into binary masks for all metrics.
- Keep preprocessing and evaluation code identical across all models for a fair comparison.
- Train the three main model variants using **Dice loss**.
- In addition, compare **Dice** and **Dice+BCE** using the **ResUNet (fine-tune all)** model.

Metrics to Report

Report the following metrics on the **test set**:

- Pixel Accuracy (ACC)
- Dice
- Intersection-over-Union (IoU)
- False Positive Rate (FP%)
- False Negative Rate (FN%)

Quantitative Comparison: Model Variants

Fill in the following table using the test-set results.

Table 1: Comparison of the three segmentation model variants. Higher is better for ACC, Dice, and IoU. Lower is better for FP% and FN%.

Model Variant	**ACC** $\uparrow$	**Dice** $\uparrow$	**IoU** $\uparrow$	**FP%** $\downarrow$	**FN%** $\downarrow$
U-Net (from scratch)					
ResUNet (encoder frozen)					
ResUNet (fine-tune all)					

Quantitative Comparison: Loss Functions

To study the effect of the loss function, train the **ResUNet (fine-tune all)** model twice:

1. once with **Dice loss**,
2. once with **Dice+BCE loss**.

   Then fill in the following table.

Table 2: Comparison of Dice and Dice+BCE losses using ResUNet (fine-tune all). Higher is better for ACC, Dice, and IoU. Lower is better for FP% and FN%.

Loss Function	**ACC** $\uparrow$	**Dice** $\uparrow$	**IoU** $\uparrow$	**FP%** $\downarrow$	**FN%** $\downarrow$
Dice					
Dice + BCE					

Visual Comparison (Required Figure)

Create one montage using the same **four representative test images**. For each image, show:

1. input image,
2. ground-truth mask,
3. U-Net prediction,
4. ResUNet (encoder frozen) prediction,
5. ResUNet (fine-tune all) prediction.

   Use the same threshold and the same visualization style for all predictions.

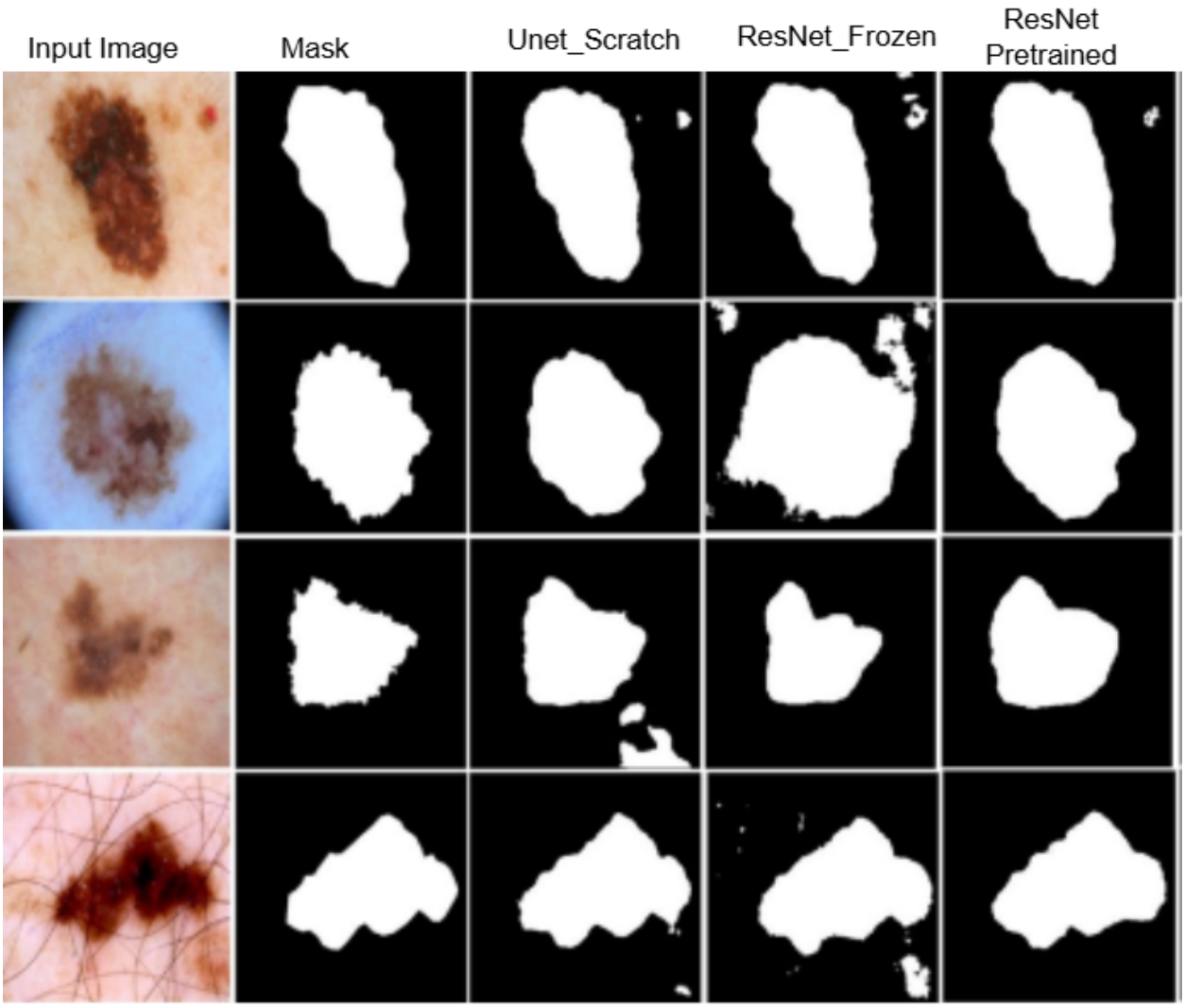

Figure 2: Visual comparison grid. Rows: different test images. Columns: input image, ground-truth mask, and predictions from the three required model variants.

Short Questions

Answer the following briefly in your report:

1. Did pretraining improve segmentation performance compared to training from scratch?
2. How did freezing the encoder compare to fine-tuning all layers?
3. Did Dice+BCE improve the results compared to Dice alone?
4. Which model produced the best visual masks?

What to Submit

Submit the following:

- your completed notebook file,
- a PDF report containing:
  - Table 1,
  - Table 2,
  - the visual comparison figure,
  - short answers to the questions above,
- any required output files requested in the notebook.

Notes

- Start early.
- Keep the notebook structure and variable names consistent with the starter code.
- Make sure your code runs from beginning to end.
- Use the same preprocessing and evaluation setup for all experiments so your comparison is fair.
- Clearly report the configuration settings used in your final experiments.